INSTITUTO TECNOLOGICO DE CULIACAN

Mauricio Lopez Lara
Carlos Ley Garcia
30/03/2026
Tópicos avanzados de IA
Tarea Bonus Modulo II

Como una introducción y antes de mostrar ejemplos de ejecución de los siguientes 4 problemas, se va a explicar en qué consiste cada ejercicio debe hacer mención que cada ejercicio se representa de manera distinta no solo por como se maneja si no también del análisis que se maneja, existen distintas áreas las cuales se aplican estos tipos de ejercicios mas que nada en la optimización y rutas incluso estos problemas son eficientes ante la optimización de recursos y rutas en ecosistemas tales como los logísticos e incluso los agrícolas, primero empezaremos por el mas sencillo de explicar y dejaremos al ultimo cual es el más complejo de entender.

1. **Árbol de expansión mínima (MST)**

Es considerado el mas sencillo por su lógica ya que es directa y visual, en este ejercicio solo hay una regla principal es no hacer un ciclo o ciclos cerrados y su objetivo claro es el conectar todo con el menor costo posible por decirlo de otro modo encontrar la mejor solución que ahorre tiempo y conecte todo esto claro dependiendo del contexto que se este dando, las decisiones que se pueden tomar de este ejercicio pueden ser paso a paso eligiendo siempre el camino mas rápido posible, por ejemplo imaginemos que necesitamos diseñar la estructura física de un sistema de comunicación en un campo universitario bueno el MST da exactamente la ruta para tirar el cableado principal conectando todos los edificios sin perder conexión de manera redundante.

2. **Problema del N reinas**

Con este ejercicio si podemos mencionar que es intermedio en cuanto a complejidad, sus reglas son fáciles de asimilar porque están basadas en un juego de mesa que es mundialmente conocido el cual pude ser fichas chinas o el mismo ajedrez, pero este problema es mas abstracto, el problema consisten que tus reinas o fichas en algunos casos no se ataquen o dicho de otro modo que no colisionen para poder ver como la computadora resuelve este problema hay que entender el concepto de prueba y error estructurado, también se podía ver como si un agente estuviera en un laberinto el agente toma un camino y de repente llega un callejon sin salida entonces no empieza el laberinto desde cero solo retrocede a su ultimo giro o camino y prueba otra dirección a eso se le conoce como una lógica que usa

el N reina la cual es llamada BackTracKing o BTK si lo quieres abreviar, el BackTracKing es una búsqueda exhaustiva sistemática que se encarga de verificar cada paso o movimiento que se da en este problema.

3. **Problema del agente viajero (TSP)**

Debo mencionar que este problema tiene algo de relación con el árbol de expansión solo que en este problema lo que se hace o lo que se quiere lograr es visitar todos los lugares y regresa al inicio usando el camino mas corto, este problema podría decirse que es engañoso y complejo, lo complicado de este problema no es el objetivo si no el porque es tan difícil el calcularlo, este requiere comprender la explosión combinatoria este termino significa que existe un numero de posibilidades o combinaciones crece de forma rápida al añadir un solo elemento nuevo y los cálculos se vuelven imposibles hasta para una computadora, imaginemos un desafío de programación de rutas de un dron y este sistema debe monitorear muchas zonas en una enorme área agrícola en Sinaloa; tienes 5 subdivisiones en un terreno o también llamadas parcelas y hay 120 rutas posibles, si tienes 15 subdivisiones podrás imaginar millones de rutas a lo que quiero llegar es que esta magnitud matemática detrás de todo este problema es simple pero es lo que lo hace complejo.

4. **Problema de programación de trabajos (JSSP)**

Este problema por mas que suene simple solo por el titulo ya que uno puede pensar que solo se trata de organizar zonas de trabajo o dar su función de manera técnica no es eso y además este problema es mas complejo que todos los anteriores por su conceptualización, modelado y su solución por sobre todo; a diferencia de los otros problema donde solo se musca una ruta o posición en el JSSP se enfrenta contra una colisión constante de restricciones múltiples y con esto vienen tres puntos importarte:

- **Dependencia estricta de orden** este como su nombre indica es que si o si se deben cumplir sus condiciones que si un trabajo tiene 5 pasos y debes

empezar desde el 1 no puedes empezar desde 2 o si no se repetirá hasta que se pueda cumplir esa condición.

- **Exclusividad de recursos** en este punto solo consiste en que si tenemos una maquina en especifico que se necesita para cierto paso pero esta siendo usado para otro trabajo completamente ajeno al paso original no puedes hacer que esa maquina trabaje rápido para que termine ese proceso debes esperar a que termine el proceso para poder continuar sí o sí.
- **Optimización del tiempo** aquí el objetivo no es solo que las cosas choquen sino que el bloque final de tiempo en que todos los trabajos terminan sea lo mas comprimido posible

Ahora quisiera hablar sobre un detalle que tiene este problema y es su efecto domino a lo que me refiero es que si ponemos un cambio por mas pequeño que sea en el comienzo de una tarea la maquina que lo esta ejecutando altera la disponibilidad que tienen el resto de las tarea creando este efecto mariposa que hace que el resto de la programación se arruine viéndolo de ese modo este es un problema estricto que una vez que lo comiences no puedes cambiarlo a la hora de su ejecución si lo quieres ver de otro modo imagina que estas intentando armar un cubo Rubik donde con el más mínimo giro todo cambia, esto requiere de visualización compleja de diagramas y el uso de heurística avanzada solo para encontrar una solución aceptable en todo ese rio porque si quieres una solución exacta siempre termina siendo inalcanzable.

Ahora que ya tenemos un contexto mas claro de como funciona cada ejercicio lo que quiero mostrar a continuación son estos ejercicios resueltos como deben ser y además de dar una breve explicación de como funcionan pero de forma mas resumida he de mencionar también que todos estos problemas fueron resueltos con Python.

1. Árbol de expansión mínima (MST)

```
Infinito = 9999999

Grafo = [

[0,2,0,6,0]

[2,0,3,8,5]

[0,3,0,0,7]

[6,8,0,0,9]

[0,5,7,9,0]

]

Nodos = 5

Seleccionados = [False,False,False,False,False]

Seleccionados[0] = True

aristas_restantes= nodos – 1

While aristas_restante > 0:

Mínimo = infinito

x=0

y=0

for i in range(nodos):

if seleccionados[i]:

for j in range(nodos):

if not seleccionados[j] and grafo[i][j] > 0

if grafo[i][j] < mínimo:

mínimo = grafo[i][j]

x=i

y=j

print (x, “-”, y, “:”, grafo [x][y])

seleccionados[y] = true

aristas_restantes = 1
```

este problema se usa una matriz de distancias donde los ceros significan que no hay conexión directa, el programa empieza primero con un nodo el cual lo marca como true en la lista que se llamó como seleccionados luego de eso se usan los for y los while para poder buscar de manera constante el numero mas pequeño o mejor dicho el camino mas corto que conecte un nodo que ya se visito con otro que aun tiene un estado de false y cuando lo encuentra lo imprime, al imprimirlo lo marca como un nodo visitado y se repite el proceso hasta conectar todos.

2. Problema del N reinas

```
N=4

Tablero = [[0,0,0,0], [0,0,0,0], [0,0,0,0], [0,0,0,0]]

Def es_seguro(tablero, fila, col):

For i in range(col):

If tablero[fila][i] ==1

Return false

For i, j in zip(range(fila,-1,-1), range(col,-1,-1)):

If tablero[i][j] == 1:

return False

for i, j in zip(range(fila,N,1), range(col,-1,-1)):

if tablero[i][j] == 1:

return False

return True

def resolver(tablero,col):

if col >= N:

return true

for i in range(N):

if es_seguro(tablero, i, col):

tablero[i][col] = 1

if resolver(tablero, col + 1):

return True

tablero[i][col] = 0
```

```
return False

if resolver(tablero, 0):

for fila in tablero:

print (fila)
```

para el problema del N reinas, en el código se creo un tablero vacio lleno de ceros esto para que la función es_seguro revise si hay un 1 o mejor dicho una reina a la izquierda o otra posición, aquí entra la función resolver intenta poner un 1 en la primera columna y luego se llama a si misma para la siguiente columna, en caso de que se atasque o se colisione se vuelve al 1 a 0 y esto es el backtranking que se menciono, y esto hace pruebas para la siguiente fila hasta que termine el problema.

3. Problema del agente viajero (TSP)

```
Import itertools

Grafo = [

[0, 10, 15, 20],

[10, 0, 35, 25],

[15, 35, 0, 30],

[20, 25, 30, 0]

]

Nodos = [0, 1, 2, 3]

Ruta_optima = None

Costo_minimo= 9999999

For permutación in itertools.permutations(nodos):

Costo_actual = 0

For i in range(len(permutacion) - 1):

Origen = permutación[i]

Destino = permutación[i+1]

Costo_actual += grafo[origen][destino]

Costo_actual += [grafo permutación[-1]][permutación][0]]

If costo_actual < costo_minimo:

Costo_minimo = costo_actual
```

```
Ruta_optima=permutación

Print(ruta_optima)

Print(costo_minimo)
```

Para este problema se implementó una herramienta la cual se llama itertools.permutations lo que hace esta herramienta es que genera todas las combinaciones posibles de las rutas lo cual es lo que estamos buscando resolver en este problema para poder determinar cual es la mejor opción, después de eso tenemos unos ciclos for los cuales calculan el cuanto se toma el recorrer cada una de las rutas sumando los valores de la matriz esto como ya se menciono anteriormente se hace para saber cual es el mejor resultado, al final de cada ruta se suma el costo de regresar al inicio, si el costo total fue menos al que se había guardado previamente entonces se actualiza y se guarda como la ruta mas optima.

4. Problema de programación de trabajos (JSSP)

```
Trabajos = [

[(0, 3), (1,2), (2,2)],

[(0, 2), (2,1), (1,4)],

[(1, 4), (2,3)]

]

Tiempo_maquinas = [0,0,0]

Tiempo_trabajos = [0,0,0]

For id_trabajo, operaciones in enumerate(trabajos):

For maquina, duración in operaciones:

Inicio = max(tiempo_trabajos[id_trabajo], tiempo_maquinas[maquina])

Fin = inicio + duración

Print(id_trabajo, maquina, inicio, fin)

Tiempo_trabajos[id_trabajo] = fin

Tiempo_maquinas[maquina] = fin

Print(max(tiempo_maquinas))
```

Para este programa nos basamos mas en que fuera un simulador básico pero secuencial, aquí tenemos una lista llamada trabajos donde cada trabajo tiene ya

tareas definidas y estas se definen por su numero de maquina y el tiempo que se tarda en trabajar, las listas son tiempo_maquinas y tiempo_trabajos estos de manera sencilla actúan como una medición de tiempo porque el programa toma una tarea cualquiera y se calcula su tiempo de inicio pero elige el valor mas grande comparando cuando se tarda una maquina en desocuparse y cuando se termina el paso anterior del trabajo, al final se suma la duración y se actualiza su tiempo, termina pasando al siguiente.

https://github.com/LeyCarlos-21170368/Topicos-de-IA.git